Architecture and Implementation

TS-Com Project Documentation Suite

Trans-Spacetime Communication Network - Undergraduate Computer Networks Course *CP352005 Networks - 4-Week Sprint*
กลุ่มที่ 2
รายชื่อสมาชิก :

นาย	นฤเศรษฐ์ อภิลักขิตพงศ์	673380044-4
นาย	คณิศร มาประจักษ์	673380031-3
นาย	สิรภัทร ลีล้าน	673380067-2
นาย	วัชรพล ดวงกองเงิน	673380290-9
นาย	ณัฐกร รุ่งฟ้า	673380512-7

File 1: architecture_spec.md

```
None
# TS-Com Network Architecture Specification v2.0
**Architectural Review Document - CP352005 Networks**

## Document Control
| Version | Date | Author | Role | Changes |
|---------|------|--------|------|---------|
| v2.0 | [Date] | [Architect Name] | Architect | Initial architectural
review |

## Team Roles
| Role | Name | Responsibilities |
|------|------|------------------|
| **Architect** | [วัชรพล ดวงกองเงิน] | การออกแบบระบบในภาพรวม, การกำหนดลำดับชั้น
(Layer), ข้อตกลงระหว่างส่วนเชื่อมต่อ (Interface contracts)|
| **Engineer** | [Name] |การติดตั้งใช้งานโปรโตคอล, การพัฒนาตัวจำลอง (Simulation)|
| **Specialist** | [ณัฐกร รุ่งฟ้า] | การวิจัยฟิสิกส์ควอนตัมและกาลเวลา, การป้องกันความขัดแย้ง
ทางเวลา (Paradox prevention)|
| **DevOps** | [Name] | การตั้งค่าสภาพแวดล้อม, ระบบ CI/CD, การรวมระบบ, การจัดทำเอกสาร
|
| **Tester/QA** | [Name] | การวางแผนการทดสอบ, การตรวจสอบความถูกต้อง, การประกัน
คุณภาพ|

---

## Part 1: Executive Summary

### 1.1 วิสัยทัศน์ของโปรเจกต์
TS-Com (Trans-Spacetime Communication Network) คือสถาปัตยกรรมเครือข่ายเชิงทฤษฎีที่ขยาย
แนวคิดเครือข่ายแบบดั้งเดิมเพื่อให้สามารถสื่อสารข้ามมิติเวลาได้
```

โปรเจกต์นี้แสดงให้เห็นถึงความเข้าใจในการออกแบบเครือข่ายแบบลำดับชั้น, โปรโตคอลการเลือกเส้นทาง, รูปแบบการระบุที่อยู่ม ความมั่นคงปลอดภัยของเครือข่าย และการป้องกัน **Paradox** ภายใต้ข้อจำกัดของรายวิชาระดับปริญญาตรี

1.2 Educational Objectives
- ประยุกต์ใช้แนวคิดของแบบจำลอง OSI/TCP-IP กับสถานการณ์รูปแบบใหม่
- ออกแบบและระบุคุณลักษณะ ของโปรโตคอลเครือข่าย
- ติดตั้งใช้งานอัลกอริทึมการเลือกเส้นทาง พร้อมการกำหนดตัวชี้วัดแบบกำหนดเอง
- สร้างแบบจำลองเครือข่าย
- ฝึกฝนการจัดทำเอกสาร ประกอบทางสถาปัตยกรรม
- ทำความเข้าใจความปลอดภัยของเครือข่าย ผ่านการเปรียบเปรยด้วยการป้องกันความขัดแย้งทางเวลา (Paradox Prevention)

1.3 Scope and Constraints

ประเด็น (Aspect)	ในขอบเขต	นอกขอบเขต
สถาปัตยกรรม (Architecture)	การออกแบบแบบลำดับชั้น, โปรโตคอล, การระบุที่อยู่	การนำไปใช้งานทางกายภาพ
การจำลอง (Simulation)	การจำลองเครือข่ายด้วยภาษา Python	การติดตั้งใช้งานบนฮาร์ดแวร์หรือระดับควอนตัม
มิติเวลา (Time Dimension)	การนำเสนอเชิงนามธรรม	การเดินทางข้ามเวลาจริง
โปรโตคอล (Protocols)	CAAP, TRP, PPP	โปรโตคอลระดับใช้งานจริง (Production-ready)
การทดสอบ (Testing)	การทดสอบระดับหน่วย (Unit tests), การทดสอบการรวมระบบ (Integration tests)	การติดตั้งใช้งานในโลกแห่งความเป็นจริง

Part 2: Architectural Review

2.1.1 Architecture Overview

ลำดับชั้น (Layer)	โปรโตคอล / ส่วนประกอบ	หน้าที่
Application	**Future-Sight, Archive-Resurrection**	แอปพลิเคชันคาดการณ์อนาคตและกู้คืนข้อมูลอดีต
Presentation	**Temporal AR, Data Serialization**	การจัดการข้อมูลเสื่อมสภาพตามเวลาและการเตรียมข้อมูลส่งออก
Session	**TSP (Timeline Sync Protocol)**	การจัดการเซสชันข้ามเส้นเวลาและการปรับจูนนาฬิกา
Transport	**PPP (Paradox Prevention Protocol)**	การตรวจจับและบล็อกความขัดแย้งทางกาลเวลา
Network	**TRP (Temporal Routing Protocol)**	การหาเส้นทางด้วย **Modified Dijkstra(Algo** ที่ใช้หาระยะที่สั้นที่สุดจาก **A to B)** ข้ามมิติเวลา
Data Link	**CAAP (Chrono-Anchor Addressing)**	การระบุที่อยู่พิกัด **4** มิติ และการแปลงชื่อ

Physical	**Quantum-Temporal Links**	การจำลองการส่งแพ็กเก็ตผ่านจุดเชื่อมต่อกาลอวกาศ

2.2 Layer-by-Layer Architecture Review

2.2.1 Physical Layer (Simulated)

Design Review Status: ✅ อนุมัติแบบมีหมายเหตุ

```
None
| Aspect      | Assessment  | Notes |
|------------|------------|-------|
| Completeness  | High |   มีส่วนเชื่อมต่อสำหรับการจำลองที่กำหนดไว้เป็นอย่างดี  |
| Feasibility   | High |   การจำลองระบบเครือข่ายด้วยภาษา Python สามารถทำได้จริง  |
| Integration | Medium |   มีส่วนเชื่อมต่อไปยัง Data Link Layer ที่ชัดเจน   |
| Innovation  | Medium |   มีการใช้เกณฑ์การวัดจุดเชื่อมต่ออิงตามเวลาแบบใหม่  |

**Interface Definition:**
```python
class PhysicalLayer:
    def transmit(self, packet, source_ca, dest_ca):
        """Transmit packet across temporal link"""
        # Simulated: returns success/failure with latency

    def receive(self, ca_address):
        """Receive packet at chrono-anchor"""
        # Simulated: returns packet if available
```
```

Risks:

- การจำลองจุดเชื่อมต่อกาลเวลา (Link Simulation) อาจมีความซับซ้อนเกินไปจนโค้ดรันไม่ออก และอาจเกิดปัญหาด้านประสิทธิภาพ (Performance) เมื่อมีการจำลองโหนดจำนวนมาก

2.2.2 Data Link Layer - Chrono-Anchor Addressing Protocol (CAAP)

Design Review Status: ✅ อนุมัติ

Address Format Specification:

รูปแบบที่อยู่ของจุดสมอเวลา (Chrono-Anchor Address) ถูกกำหนดให้มีขนาดรวมทั้งสิ้น 24 ไบต์ เพื่อให้ครอบคลุมข้อมูลที่จำเป็นในการระบุและอ้างอิงตำแหน่งของจุดสมอเวลาแต่ละจุดในระบบอย่างชัดเจน โดยที่อยู่ดังกล่าวถูกแบ่งออกเป็นฟิลด์ย่อยที่มีขนาดฟิลด์ละ 4 ไบต์ ดังรายละเอียดต่อไปนี้:

```None
Chrono-Anchor Address (24 bytes total):
┌──────────────────────────────────────┐
│ Field            | Size  | Description   |
├──────────────────────────────────────┤
| Spatial X        | 4B    | x-coordinate  |
| Spatial Y        | 4B    | y-coordinate  |
| Spatial Z        | 4B    | z-coordinate  |
| Temporal T       | 4B    | time value    |
| Gravity Potential| 4B    | gravitational |
| Metadata         | 4B    | flags, version|
└──────────────────────────────────────┘
```

Address Resolution:

```Python
class CAAP:
    def resolve_address(self, logical_name):
        """Map logical name to CA address"""

    def validate_address(self, ca_address):
        """Ensure address format is valid"""

    def calculate_temporal_distance(self, ca1, ca2):
        """Return |Δt| between anchors"""
```

Review Notes:

ความแม่นยำและการขยายตัว (Scalability & Precision):

- รูปแบบที่อยู่ขนาด 24 ไบต์ รองรับมิติที่จำเป็นครบถ้วน รวมถึง **Gravity Potential** ซึ่งมีความสำคัญอย่างยิ่งต่อการรักษาความเสถียรของแพ็กเก็ตเมื่อเดินทางผ่านโซนที่มีแรงโน้มถ่วงต่างกัน
- อย่างไรก็ตาม เนื่องจากปัจจุบันรองรับได้เพียง 5 โหนด ทีมพัฒนาควรคำนึงถึงขีดจำกัดในการขยายตัว (Scalability) ในอนาคตหากต้องรองรับโหนดจำนวนมหาศาลข้ามศตวรรษ

เอกลักษณ์ของโหนดในทุกเส้นเวลา (Temporal Uniqueness):

- ทุกหน่วยในเครือข่ายต้องมีตัวระบุที่ไม่ซ้ำกัน ไม่ใช่แค่ในเชิงพื้นที่ แต่ต้องไม่ซ้ำกันในมิติเวลา เพื่อป้องกันความสับสนของที่อยู่เมื่อเกิดการเชื่อมต่อข้าม **Timeline** คู่ขนาน

กลไกการแปลงที่อยู่และประสิทธิภาพ (Resolution & Caching):

- การใช้ **Address Caching** เป็นสิ่งจำเป็นเพื่อลด **Latency** ในการหาเส้นทางข้ามมิติเวลาที่มีความหน่วงสูง (High Latency)

- กลไกการแปลงชื่อ (Logical-to-Physical) ต้องมีความปลอดภัยสูงเพื่อป้องกันการโจมตีประเภท **Address Spoofing** จากเส้นเวลาที่ไม่น่าเชื่อถือ (**Untrusted Time**)

ความสมบูรณ์ของส่วนต่อประสาน (Interface Integrity):

- ส่วนต่อประสานระหว่างชั้น **Data Link** และ **Network Layer** ได้รับการกำหนดไว้อย่างชัดเจนผ่านฟังก์ชัน calculate_temporal_distance ซึ่งช่วยให้การคำนวณค่า Cost ในชั้นถัดไปมีความแม่นยำ

การรายงานข้อผิดพลาดและการควบคุมการไหล (Error Handling & Flow Control):

- **ข้อควรระวัง:** ระบบต้องเพิ่มกลไกรายงานข้อผิดพลาดหากเกิดสภาวะ **Address Drift** (พิกัดเวลาเคลื่อน) หรือ **Resolution Timeout** เมื่อไม่สามารถติดต่อจุดสมอเวลาปลายทางได้ภายในเวลาที่กำหนด (เช่น 30 วินาทีในระบบจำลอง)

2.2.3 Network Layer - Temporal Routing Protocol (TRP)

Design Review Status: ✅ อนุมัติ

- **α = 0.5 : ค่าน้ำหนักความสำคัญของระยะห่างเวลา (Temporal displacement)**
- **β = 0.3 : ค่าน้ำหนักความสำคัญของต้นทุนพลังงาน (Energy cost)**
- **γ = 0.2 : ค่าน้ำหนักความสำคัญของความเสี่ยงต่อ Paradox (Paradox risk)**

Routing Algorithm Design:

```python
class TRP:
    def __init__(self):
        self.routing_table = {}
        self.topology = {}

    def calculate_route(self, source, destination):
        """
        Modified Dijkstra with temporal constraints:
        Cost = 0.5Δt + 0.3ΔE + 0.2ParadoxRisk
        where:
        - Δt = temporal displacement
        - ΔE = energy cost
        - ParadoxRisk = probability of paradox
        - α,β,γ = weighting factors
        """
```

```python
    def update_routing_table(self, topology_update):
        """Handle topology changes"""

    def forward_packet(self, packet):
        """Determine next hop"""
```

กลไกการรักษาเสถียรภาพของเครือข่าย:

- **ช่วงเวลาการอัปเดต (Update Interval):** กำหนดให้รีเฟรชตารางการเลือกเส้นทางทุกๆ **30 วินาที (ในเวลาจำลอง)** เพื่อป้องกันข้อมูลล้าสมัย
- **การวิเคราะห์การลู่เข้า (Convergence Analysis):** เนื่องจาก Dijkstra เป็นอัลกอริทึมที่ให้ผลลัพธ์ที่เสถียร (Stable) ในกราฟที่มีค่าน้ำหนักเป็นบวก ระบบจึงรับประกันการลู่เข้าของข้อมูลภายในเวลาไม่เกิน 0.5 วินาทีต่อการตัดสินใจ
- **ระบบแคชเส้นทาง (Route Caching):** ติดตั้งกลไกการเก็บข้อมูลเส้นทางที่คำนวณแล้วไว้ใน routing_cache = {} เพื่อลดภาระการประมวลผลซ้ำซ้อนและเพิ่มความเร็วในการส่งแพ็กเก็ต

2.2.4 Transport Layer - Paradox Prevention Protocol (PPP)

Design Review Status: ✅ อนุมัติ

กฎการตรวจจับ: บล็อก **Grandfather Paradox** (R001) และบล็อก **Information Loop** เมื่อเกิน 10 Hops (R002)
การทำงาน: มีระบบ **Paradox Probability Scoring** เพื่อประเมินความเสี่ยงก่อนส่ง

Implementation:

```python
class ParadoxFilter:
    def __init__(self):
        self.rules_engine = RulesEngine()
        self.violation_log = []

    def validate_packet(self, packet, timeline_context):
        """
        Returns:
            - ACCEPT: packet can proceed
            - BLOCK: packet must be dropped
            - DEFER: requires human review
        """
```

```
    def calculate_paradox_score(self, packet):
        """0.0 (safe) to 1.0 (certain paradox)"""
```

Review Notes:

- แนวทางการตรวจสอบแบบอิงกฎ (Rule-based approach) มีความเหมาะสมสำหรับการจำลองระบบ (Simulation) เนื่องจากสามารถกำหนดเงื่อนไขและควบคุมพฤติกรรมของระบบได้อย่างชัดเจน
- การกำหนดให้มีการตรวจสอบโดยมนุษย์ (Human review) ในกรณีพิเศษถือเป็นแนวทางที่เหมาะสมสำหรับการจัดการสถานการณ์ขอบเขต (Edge cases) ที่ระบบอัตโนมัติอาจตัดสินใจได้ไม่สมบูรณ์
- ควรพิจารณาเพิ่มระบบการคำนวณ **คะแนนความน่าจะเป็นของ Paradox (Paradox Probability Scoring)** เพื่อช่วยประเมินระดับความเสี่ยงและสนับสนุนการตัดสินใจของระบบให้มีความแม่นยำมากยิ่งขึ้น

2.2.5 Session Layer - Timeline Synchronization Protocol (TSP)

Design Review Status: ✅ อนุมัติ
จุดเด่น : จะต้องมีระบบ **Synchronize Clocks** เพื่อแก้ปัญหา **Temporal Drift** และแอปพลิเคชันรองรับการกู้คืนข้อมูลประวัติศาสตร์

Session Management:

```python
class TimelineSession:
    def __init__(self, session_id, source_t, dest_t):
        self.session_id = session_id
        self.source_timeline = source_t
        self.dest_timeline = dest_t
        self.state = "ESTABLISHING"

    def synchronize_clocks(self):
        """Adjust for temporal drift"""

    def heartbeat(self):
        """Maintain session across time"""

    def close(self):
        """Terminate temporal session"""
```

Session States:

```
None
CLOSED → ESTABLISHING → SYNCHRONIZED → ACTIVE → CLOSING → CLOSED
                                ↳ FAILED ────────┘
```

2.2.6 Presentation Layer - Temporal AR Interface

Design Review Status: ✅ อนุมัติ

Data Serialization:

```python
class TemporalSerializer:
    def encode_temporal_data(self, data, source_time):
        """Prepare data for temporal transmission"""

    def decode_temporal_data(self, encoded_data, target_time):
        """Reconstruct data for target timeline"""

    def handle_temporal_drift(self, data, time_delta):
        """Adjust for time-based data degradation"""
```

2.2.7 Application Layer - Demo Applications

Design Review Status: ✅ อนุมัติ

Application Suite:

1. **Future-Sight Commander**

    - ใช้ข้อมูลจากอนาคตเพื่อคาดการณ์ผลลัพธ์ที่อาจเกิดขึ้น
    - แสดงผลในรูปแบบเครื่องมือสนับสนุนการตัดสินใจ (Decision Support Visualization) เพื่อช่วยผู้ใช้งานวิเคราะห์สถานการณ์และเลือกแนวทางที่เหมาะสม

2. **Archive Resurrector**

    - ใช้สำหรับกู้คืนข้อมูลทางประวัติศาสตร์ที่สูญหายหรือไม่สามารถเข้าถึงได้
    - มีอินเทอร์เฟซสำหรับการสำรวจเส้นเวลา (Timeline Browser Interface) เพื่อค้นหาและเรียกดูข้อมูลจากช่วงเวลาต่าง ๆ

3. **Medical Loopback**

    - ส่งสูตรวัคซีนหรือข้อมูลทางการแพทย์ย้อนกลับไปยังอดีตผ่านเครือข่ายเชิงเวลา
    - ใช้เป็นการจำลองแนวคิดการป้องกันการแพร่ระบาดล่วงหน้า (Pandemic Prevention Simulation)

2.3 Interface Contracts

Cross-Layer Interfaces:

```
None
Layer N → Layer N-1:
    request(service_type, parameters)

Layer N-1 → Layer N:
    confirm(status, result)
    indicate(event, data)
```

Example: Network → Data Link:

```python
# Network layer requests address resolution
data_link.request("RESOLVE", {
    "logical_name": "CA_PAST_001",
    "timeout": 30
})

# Data link responds
data_link.confirm("SUCCESS", {
    "ca_address": "CA[10,20,5,2025,0.1]",
    "resolution_time": 0.5
})
```

2.4 Non-Functional Requirements Review

Requirement	Target	Current	Status
Scalability	Support 10+ nodes	5 nodes	⚠️ Needs work
Performance	<1s routing decision	<0.5s	✅ Good
Reliability	95% packet delivery	Simulated	⚠️ Needs test
Maintainability	Modular layers	Good	✅ Good
Testability	Unit test coverage >80%	Planned	🔄 In progress

2.5 Security Architecture (Paradox Prevention)

Security Zones:

```
None
┌──────────────────┐
│ Trusted Timeline│  Current timeline (user's present)
├──────────────────┤
│ Temporal DMZ    │  Packet inspection, paradox filtering
├──────────────────┤
│ Untrusted Time  │  Past/future timelines (risk assessed)
└──────────────────┘
```

การป้องกัน Paradox ในฐานะการเปรียบเทียบกับความมั่นคงปลอดภัยของเครือข่าย

Firewall → กฎการกรอง Paradox (Paradox Filter Rules)
ทำหน้าที่ตรวจสอบและกรองแพ็กเก็ตที่อาจก่อให้เกิดความขัดแย้งทางกาลเวลา ก่อนที่จะอนุญาตให้เข้าสู่ระบบหรือส่งต่อไปยังเส้นเวลาปลายทาง

Intrusion Detection System (IDS) → การตรวจจับการละเมิดเหตุและผล (Causality Violation Detection)
ตรวจจับพฤติกรรมที่ผิดปกติ เช่น ข้อมูลที่มาถึงก่อนเวลาที่ควรถูกสร้างขึ้น หรือเหตุการณ์ที่ไม่สอดคล้องกับลำดับเหตุและผล

Access Control → การกำหนดสิทธิ์ของเส้นเวลา (Timeline Permissions)
ควบคุมสิทธิ์การเข้าถึงระหว่างเส้นเวลาต่าง ๆ เพื่อป้องกันการส่งข้อมูลไปยังช่วงเวลาที่ไม่ได้รับอนุญาต

Audit Log → การบันทึกการละเมิด (Violation Logging)
บันทึกเหตุการณ์ที่เกี่ยวข้องกับการละเมิดกฎ Paradox เพื่อใช้ในการตรวจสอบ วิเคราะห์ และปรับปรุงความปลอดภัยของระบบในอนาคต

Part 3: Architecture Decisions Log

Decision 1: Python-Based Simulation

Date: 18/2/2026
Decision: Use Python with NetworkX for simulation
Rationale: Fast development, good visualization, team familiarity
Alternatives Considered: NS-3, OMNeT++ (too complex for timeframe)
Status: ✅ Approved

Decision 2: 4D Addressing Scheme

Date: 20/2/2026
Decision: Use (x,y,z,t,gravity) 24-byte address format
Rationale: Extensible, includes all required dimensions
Alternatives Considered: IPv6 extension (insufficient)
Status: ✅ Approved

Decision 3: Weighted Cost Routing

Date: 21/2/2026
Decision: Multi-metric routing (time, energy, paradox)
Rationale: Demonstrates complex routing decisions
Alternatives Considered: Single metric (Δt only)
Status: ✅ Approved

Part 4: Architectural Review Sign-off

Role	Name	Signature	Date	Comments
Architect	Vacharapoln D.	Vacharapoln	22/2/2026	No comment.
Engineer	[Name]			
Specialist	Nattakron R.	Nattakron	22/2/2026	The time paradox turbine has melted down. Architect manager, please assign Engineer agents to a coolant section ASAP
DevOps	[Name]			
Tester/QA	Siraphat L.	Siraphat	22/2/2026	Performed basic unit and integration testing, and reported them clearly.

Review Outcome: ✅ อนุมัติ

Appendices

Appendix A: Glossary

Term	Definition
Chrono-Anchor	Temporal router/node with 4D address
Temporal Displacement	Time difference between nodes
Paradox Risk	Probability of causality violation
Timeline	Linear sequence of events

Appendix B: References

1. Kurose & Ross, "Computer Networking: A Top-Down Approach"
2. Tanenbaum, "Computer Networks"
3. Novikov Self-Consistency Principle
4. OSI Model - ISO/IEC 7498-1

This architectural specification is approved for the CP352005 Networks undergraduate project.

```
None

---

# File 2: implementation_plan.md

```markdown
# TS-Com Implementation Plan v2.0
**Implementation Analysis & 4-Week Sprint Planning - CP352005 Networks**

## Document Control
| Version | Date | Author | Role | Changes |
|---------|------|--------|------|---------|
| v2.0 | [Date] | [Name] | DevOps | Initial implementation analysis |

## Team Role Assignment

| Role | Assigned To | Primary Responsibilities | Secondary Responsibilities
|
```
```

```
|------|------------|------------------------|--------------------------|
| **Architect** | [Name] | System design, interface contracts, architecture
decisions | Code review, documentation |
| **Engineer** | [Name] | Protocol implementation, core algorithms,
simulation | Testing support, bug fixes |
| **Specialist** | [Name] | Paradox rules, temporal physics, domain
expertise | Documentation, validation |
| **DevOps** | [Name] | Environment, CI/CD, integration, repo management |
Documentation, deployment |
| **Tester/QA** | [Name] | Test plans, test cases, quality metrics,
validation | Bug tracking, reporting |

---

## Part 1: Implementation Analysis

### 1.1 Complexity Assessment

| Component | Complexity (1-5) | Risk Level | Estimated Effort (hours) |
|----------|----------------|----------|----------------------|
| CAAP (Addressing) | 3 | Low | 8-10 |
| TRP (Routing) | 5 | High | 20-25 |
| PPP (Paradox) | 4 | Medium | 12-15 |
| TSP (Session) | 3 | Low | 8-10 |
| Simulation Framework | 4 | Medium | 15-20 |
| Visualization | 3 | Low | 6-8 |
| Testing Suite | 3 | Medium | 10-12 |
| Documentation | 2 | Low | Continuous |

**Total Estimated Effort:** 79-100 person-hours
**Available Team Hours (4 weeks × 5 members × 6 hours/week):** 120 hours
**Buffer:** 20-40 hours (16-33%)

### 1.2 Dependency Analysis
```

```
Week 1        Week 2        Week 3        Week 4
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│ Architecture │→│ Protocols  │→│ Integration │→│ Final Demo  │ │ Design      │ │
│ Implementation │ │ & Testing  │ │ & Delivery  │ └──────────────────┘
└──────────────────┘  └──────────────────┘  └──────────────────┘  ↓           ↓
↓           ↓ CAAP Spec → CAAP Code → CAAP Test → CAAP Integrated TRP Spec →
TRP Code → TRP Test → TRP Integrated PPP Spec → PPP Code → PPP Test → PPP
Integrated Sim Setup → Sim Dev → Sim Test → Sim Final
```

None

```
**Critical Path:** TRP Implementation → Integration → Testing
**Parallelizable Tasks:** CAAP, PPP, Visualization, Documentation

### 1.3 Technical Debt Assessment

| Potential Debt | Impact | Mitigation Strategy |
|----------------|--------|---------------------|
| Routing algorithm too complex | High | Start simple, iterate |
| Simulation performance | Medium | Profile early, optimize later |
| Integration friction | High | Daily integration, interface contracts |
| Documentation gaps | Medium | Document as you code |

---

## Part 2: 4-Week Sprint Planning

### Week 1: Foundation Sprint (Days 1-5)
**Theme:** Architecture, Setup, and Component Design

#### Day 1: Kickoff & Environment Setup
| Time | Activity | Lead | Participants |
|------|----------|------|--------------|
| 9:00-10:00 | Sprint Planning | DevOps | All |
| 10:00-12:00 | Environment Setup | DevOps | All |
| 13:00-15:00 | Architecture Review | Architect | All |
| 15:00-17:00 | Role-specific tasks | Each | Individual |

**Deliverables:**
- [x] GitHub repository created (DevOps)
- [x] Development environment documented (DevOps)
- [x] Requirements installed (All)
- [x] Initial architecture diagram (Architect)

**Role Tasks:**
| Role | Tasks |
|------|-------|
| Architect | Finalize layer interfaces, create interface contracts |
| Engineer | Review Python/NetworkX documentation, prototype simple graph |
| Specialist | Research paradox rules, draft 5 core rules |
| DevOps | Setup CI (GitHub Actions), create project board |
| Tester/QA | Create test plan template, define test categories |

#### Day 2: Addressing & Physical Layer Design
**Focus:** CAAP specification and link simulation

**Pair Programming:**
- Architect + Engineer: Address format design
```

```
- Specialist + Tester: Paradox rule documentation

**Deliverables:**
- [ ] CAAP address format specification (Architect)
- [ ] Address resolution algorithm (Engineer)
- [ ] Link simulation prototype (Engineer)
- [ ] Paradox rule v0.1 (Specialist)
- [ ] Test cases for addressing (Tester)

#### Day 3: Routing Protocol Design
**Focus:** TRP algorithm design

**Deliverables:**
- [ ] TRP routing algorithm pseudo-code (Engineer)
- [ ] Cost function definition (Architect + Specialist)
- [ ] Routing table structure (Engineer)
- [ ] Test cases for routing (Tester)

**Key Decision:** Finalize weighting factors (α,β,γ) for routing cost

#### Day 4: Paradox Prevention Design
**Focus:** PPP rule engine

**Deliverables:**
- [ ] Complete paradox rule set (Specialist)
- [ ] Rule engine design (Engineer)
- [ ] Validation interface (Architect)
- [ ] Test cases for paradox scenarios (Tester)

#### Day 5: Week 1 Review & Integration
**Activities:**
- 14:00-15:00: Code review session
- 15:00-16:00: Integration testing
- 16:00-17:00: Sprint review & Week 2 planning

**Week 1 Success Criteria:**
- [ ] All specifications documented
- [ ] Development environment working for all
- [ ] Basic simulation framework created
- [ ] Test framework established
- [ ] CI passing on main branch

---

### Week 2: Implementation Sprint (Days 6-10)
**Theme:** Core Protocol Implementation

#### Day 6-7: CAAP & Physical Layer Implementation
```

```
**Lead:** Engineer
**Support:** Architect (code review), Tester (test cases)

**Tasks:**
1. Implement ChronoAnchorAddress class
2. Create address resolution module
3. Build link simulation with temporal metrics
4. Write unit tests (minimum 80% coverage)

**Definition of Done:**
- [ ] Address creation and validation working
- [ ] Resolution returns correct addresses
- [ ] Link simulation calculates Δt correctly
- [ ] All tests passing

#### Day 8-9: TRP Implementation
**Lead:** Engineer
**Support:** Specialist (routing metrics), Tester (test scenarios)

**Tasks:**
1. Implement routing table management
2. Code modified Dijkstra with temporal weights
3. Create route calculation module
4. Add route caching mechanism

**Critical Path Item:** Must be completed by Day 9 EOD

#### Day 10: PPP Implementation
**Lead:** Specialist + Engineer
**Support:** Tester (validation)

**Tasks:**
1. Implement paradox rule engine
2. Create packet validation pipeline
3. Add violation logging
4. Test with paradox scenarios

**Week 2 Success Criteria:**
- [ ] All core protocols implemented
- [ ] Unit tests passing (>80% coverage)
- [ ] Individual components run independently
- [ ] Documentation updated

---

### Week 3: Integration Sprint (Days 11-15)
**Theme:** Integration, Testing, and Refinement
```

```
#### Day 11: Component Integration - Phase 1
**Integration Order:**
1. CAAP + Physical Layer
2. Add TRP
3. Add PPP

**Integration Lead:** DevOps
**Testing Lead:** Tester

**Activities:**
- Morning: Integrate CAAP + Physical
- Afternoon: Add TRP, test routing
- Evening: Run integration tests

#### Day 12: Component Integration - Phase 2
**Integration Tasks:**
1. Add PPP to stack
2. Implement end-to-end packet flow
3. Create test harness

**Milestone:** First end-to-end packet transmission by EOD

#### Day 13: System Testing & Bug Fixes
**Testing Activities:**

| Test Suite | Owner | Target |
|------------|-------|--------|
| Unit Tests | Engineer | Re-run all |
| Integration Tests | Tester | 20 scenarios |
| Performance Tests | DevOps | Response time <1s |
| Paradox Scenarios | Specialist | 10 edge cases |

#### Day 14: Visualization & Demo Development
**Lead:** Engineer
**Support:** All

**Visualization Requirements:**
- Network topology graph (x vs t)
- Packet flow animation
- Routing decision display
- Paradox detection alerts

#### Day 15: Week 3 Review & Dry Run
**Activities:**
- 10:00-12:00: Full system test
- 13:00-15:00: Bug fixing
- 15:00-17:00: Internal demo dry run
```

```
**Week 3 Success Criteria:**
- [ ] All components integrated
- [ ] End-to-end packet flow working
- [ ] 3 demo scenarios functional
- [ ] Visualization displays correctly
- [ ] Test coverage >80%

---

### Week 4: Delivery Sprint (Days 16-20)
**Theme:** Finalization, Documentation, and Presentation

#### Day 16: Documentation Sprint
| Document | Owner | Template |
|----------|-------|----------|
| User Guide | DevOps | `docs/user_guide.md` |
| API Reference | Engineer | `docs/api.md` |
| Test Report | Tester | `docs/test_report.md` |
| Architecture Final | Architect | `architecture_spec.md` |
| Implementation Summary | All | `README.md` |

#### Day 17: Polish & Optimization
**Focus Areas:**
- Code cleanup and comments
- Performance optimization
- Edge case handling
- Visualization enhancement

#### Day 18: Presentation Development
**Tasks:**
1. Create slide deck (All)
2. Record demo video (Engineer)
3. Prepare live demo script (Tester)
4. Rehearse presentation (All)

**Presentation Structure:**
| Section | Duration | Presenter |
|---------|----------|----------|
| Introduction | 2 min | Architect |
| Architecture | 3 min | Architect |
| Implementation | 4 min | Engineer |
| Paradox Prevention | 3 min | Specialist |
| Demo | 5 min | Engineer |
| Testing Results | 2 min | Tester |
| Conclusion | 1 min | DevOps |

#### Day 19: Final Review & Rehearsal
**Schedule:**
```

```
- 9:00-11:00: Final code review
- 11:00-12:00: Documentation review
- 13:00-15:00: Presentation rehearsal
- 15:00-16:00: Feedback & fixes
- 16:00-17:00: Final adjustments

#### Day 20: Submission Day
**Final Checklist:**

**Code:**
- [ ] All code pushed to main branch
- [ ] Tagged as `v1.0.0`
- [ ] README updated with setup instructions
- [ ] Requirements.txt complete

**Documentation:**
- [ ] Architecture specification
- [ ] Implementation plan
- [ ] User guide
- [ ] API documentation
- [ ] Test report

**Presentation:**
- [ ] Slide deck (PDF)
- [ ] Demo video (MP4)
- [ ] Live demo ready

**Submission Package:**
- [ ] GitHub repository link
- [ ] All deliverables zipped
- [ ] Team contribution statement
- [ ] Peer evaluations

---

## Part 3: Role-Specific Implementation Analysis

### 3.1 Architect's Implementation Analysis

**Key Concerns:**
- Interface stability between layers
- Protocol compatibility
- Scalability of addressing scheme

**Implementation Checklist:**
- [ ] All layer interfaces documented
- [ ] Address format validated with test cases
- [ ] Protocol message formats specified
```

```
- [ ] Architecture decisions logged

**Risk Mitigation:**
- Create interface mocks for parallel development
- Review all code against architecture spec
- Track architectural drift

### 3.2 Engineer's Implementation Analysis

**Core Implementation Tasks:**
```

Priority 1 (Must Have): ├── ChronoAnchorAddress class ├── RoutingTable management ├── Dijkstra with temporal weights ├── Packet structure └── Basic simulation

Priority 2 (Should Have): ├── Route caching ├── Multi-path routing ├── Visualization └── Performance optimization

Priority 3 (Nice to Have): ├── Load balancing ├── Advanced metrics ├── GUI interface └── Real-time updates

```
None

**Technical Challenges:**
- Efficient routing in temporal dimension
- Simulation performance with 10+ nodes
- Integration with visualization

### 3.3 Specialist's Implementation Analysis

**Domain Rules Implementation:**

| Rule | Implementation Complexity | Validation Method |
|------|---------------------------|-------------------|
| Grandfather Paradox | Medium | Unit tests with scenarios |
| Information Loop | Low | Hop count tracking |
| Causality Violation | High | Timeline consistency checks |
| Temporal Consistency | Medium | State validation |

**Research Requirements:**
- [ ] Review Novikov self-consistency principle
- [ ] Document 5 paradox scenarios
- [ ] Create paradox probability model
- [ ] Define edge cases

### 3.4 DevOps's Implementation Analysis
```

```
**Infrastructure Requirements:**

| Component | Technology | Configuration |
|-----------|------------|---------------|
| Version Control | GitHub | Branch protection |
| CI/CD | GitHub Actions | Python 3.8+ |
| Documentation | MkDocs | Material theme |
| Testing | pytest | Coverage reporting |
| Dependencies | pip | requirements.txt |

**CI Pipeline:**
```yaml
name: TS-Com CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Setup Python
        uses: actions/setup-python@v2
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest tests/ --cov=src
      - name: Check style
        run: flake8 src/
```

3.5 Tester/QA's Implementation Analysis

Test Strategy:

Test Levels:

1. **Unit Tests** (Week 2)

    - Individual class testing
    - Method-level validation
    - Expected: 50+ test cases

2. **Integration Tests** (Week 3)

    - Layer interaction testing
    - End-to-end packet flow
    - Expected: 20+ scenarios
```

3. **System Tests** (Week 4)

    - Full simulation testing
    - Performance benchmarks
    - Expected: 10+ test runs

Test Matrix: | Component | Unit Tests | Integration | System | Owner |
|----------|----------|----------|---------|-------| | CAAP | 15 | 5 | 2 | Tester | | TRP | 20 | 8 | 3 | Tester | | PPP | 15 | 5 | 3 | Tester | | Simulation | 10 | 2 | 2 | Tester |

Bug Tracking Template:

```
None
## Bug Report
**ID:** TSCOM-[Number]
**Severity:** Critical/High/Medium/Low
**Component:** [CAAP/TRP/PPP/Sim]
**Description:**
**Steps to Reproduce:**
**Expected Behavior:**
**Actual Behavior:**
**Assigned To:**
**Status:** Open/In Progress/Resolved/Verified
```

Part 4: Daily Standup Template

```
None
# Daily Standup - Day [X] - [Date]

## What I did yesterday:
- Architect:
- Engineer:
- Specialist:
- DevOps:
- Tester:

## What I'll do today:
- Architect:
- Engineer:
- Specialist:
- DevOps:
- Tester:
```

```
## Blockers:
1. [Blocker - Owner]
2. [Blocker - Owner]

## Progress Metrics:
- [ ] Tests passing: [X]/[Y]
- [ ] Components integrated: [X]/5
- [ ] Documentation complete: [X]%
- [ ] Days until deadline: [X]
```

Part 5: Sprint Review Checklist

End of Sprint 1 (Week 1)

- ☑ System Architecture Approved
- ☑ Core Concepts Defined
- ☑ Development Environment Working
- ☑ Test Framework Ready
- ☑ Initial Risk & Ethics Review Completed
- ☑ Team Velocity Measured

End of Sprint 2 (Week 2)

- ☑ Core Temporal Protocols Implemented
- ☑ Causal / Paradox Filter AI Integrated (Prototype)
- ☑ Unit Tests Passing
- ☑ Code & Model Reviewed

End of Sprint 3 (Week 3)

- ☑ All components integrated
- ☑ End-to-End Communication Flow Working
- ☑ Demo scenarios ready
- ☑ Visualization & Interface Functional
- ☑ System Stability Reviewed

End of Sprint 4 (Week 4)

- ☑ All Deliverables Completed
- ☑ Documentation Complete
- ☑ Presentation prepared
- ☑ Submission package assembled

Part 6: Contingency Plans

Plan A: Behind Schedule

If behind by >2 days:

1. Reduce scope (remove nice-to-have features)
2. Increase pair programming
3. Extend working hours (max +2 hours/day)

Plan B: Integration Failure

If integration fails:

1. Roll back to last working version
2. Isolate problematic component
3. Create simplified interface
4. Re-integrate with mocks

Plan C: Team Member Absence

If member absent >2 days:

1. Redistribute tasks
2. Pair programming covers gaps
3. Adjust deliverables
4. Document knowledge transfer

Plan D: Technical Block

If technical issue blocks progress:

1. 2-hour investigation limit
2. Escalate to team
3. Seek instructor help
4. Implement workaround

Part 7: Success Criteria Sign-off

Criteria	Target	Current	Owner	Status
Architecture approved	Week 1		Architect	Done
All components implemented	Week 2		Engineer	⏳
Integration complete	Week 3		DevOps	⏳
Tests passing (>80% coverage)	Week 4		Tester	⏳
Demo ready	Week 4		All	⏳
Documentation complete	Week 4		All	⏳
Presentation prepared	Week 4		All	⏳

Approval

Role	Name	Signature	Date
Architect	[Name]	Vacharapoln	
Engineer	[Name]		
Specialist	Nattakron R.	Nattakron	
DevOps	[Name]		
Tester/QA	Kanisorn M.	Kanisorn	

This implementation plan is approved for the CP352005 Networks undergraduate project. All team members agree to follow this plan and report deviations immediately.

Plan Version: v2.0
Last Updated: [Date]
Next Review: Daily standups, Weekly sprints

None

These two documents provide a complete, role-assigned framework for the 4-week undergraduate project. The **architecture_spec.md** includes the architectural review with conditional approvals and required changes, while the **implementation_plan.md** provides detailed role-based analysis, daily schedules, and contingency planning. Both documents are ready for team use and instructor review.

Roles and Responsibility

TS-Com Project: Roles, Responsibilities & Boundaries Matrix

Team Role Assignment Table

Role	Assigned To	Primary Responsibilities	Secondary Responsibilities	Decision Authority	Reporting To
Architect	Vacharapoln	• System-wide design decisions • Layer interface definitions • Protocol specifications • Architecture documentation	• Code review approval • Technical debt tracking • Design pattern guidance	• Final say on architecture • Interface changes • Technology stack selection	Instructor
Engineer	[Name]	• Core protocol implementation • Algorithm development • Simulation coding • Performance optimization	• Bug fixes • Code documentation • Technical prototyping	• Implementation approach • Code structure • Tool/library selection	Architect
Specialist	Nattakron	• Paradox rule definitions • Temporal physics research	• Technical documentation • Validation scenarios	• Paradox rule logic • Domain-specific decisions	Architect

Role	Assigned To	Primary Responsibilities	Secondary Responsibilities	Decision Authority	Reporting To
		• Domain expertise • Edge case identification	• Specialist testing	• Research direction	
DevOps	[Name]	• Environment setup • CI/CD pipeline • Version control management • Integration coordination	• Documentation hosting • Build automation • Deployment scripts	• Tooling choices • Branch strategy • Integration schedule	Engineer
Tester/QA	[Name]	• Test strategy development • Test case creation • Bug tracking • Quality metrics	• User acceptance testing • Performance testing • Documentation review	• Test coverage requirements • Quality gates • Release readiness	All

---

Detailed Responsibility Matrix

By Project Phase

Phase	Architect	Engineer	Specialist	DevOps	Tester/QA
Week 1: Foundation	Design layer interfaces Create architecture diagrams	Research Python/NetworkX	Research paradox rules Document 5 core rules	Create GitHub repo Setup CI pipeline	Create test plan template Define test categories

Phase	Architect	Engineer	Specialist	DevOps	Tester/QA
	Define protocol specs	Setup development environment Create proof-of-concept	Define temporal concepts	Create project board	Setup bug tracker
Week 2: Implementation	Review code against spec Resolve design issues Update architecture docs	Implement CAAP addressing Code TRP routing Develop PPP rules engine	Validate paradox rules Create test scenarios Document edge cases	Manage branches Coordinate integration Build automation	Write unit tests Create test data Run daily tests
Week 3: Integration	Verify interface compliance Sign off on integration Architecture validation	Integrate components Fix integration bugs Optimize performance	Test paradox scenarios Validate temporal logic Review edge cases	Lead integration sessions Maintain build stability Document deployment	Run integration tests Track regression bugs Generate test reports
Week 4: Delivery	Final architecture review Sign off deliverables Present architecture	Final code cleanup Create demo video Optimize final build	Validate all scenarios Final domain review Contribute to presentation	Package deliverables Freeze repository Create submission ZIP	Final test execution Generate coverage report Verify all requirements

---

Responsibility Area Matrix

By Component

Component	Design Owner	Implementation Owner	Testing Owner	Documentation Owner
CAAP (Addressing)	Architect	Engineer	Tester/QA	Architect
TRP (Routing)	Architect + Specialist	Engineer	Tester/QA	Engineer
PPP (Paradox)	Specialist	Engineer + Specialist	Tester/QA + Specialist	Specialist
TSP (Session)	Architect	Engineer	Tester/QA	Engineer
Simulation Framework	Architect	Engineer	Tester/QA	DevOps
Visualization	Architect	Engineer	Tester/QA	DevOps
Testing Framework	Tester/QA	Tester/QA + DevOps	Tester/QA	Tester/QA
Documentation	All	All	Tester/QA	DevOps

Decision Authority Matrix

Decision Type	Architect	Engineer	Specialist	DevOps	Tester/QA	Instructor
Architecture changes	**Approve**	Consult	Consult	Consult	Consult	Final if disputed
Technology selection	Propose	**Approve**	Consult	Consult	Consult	Review
Protocol design	**Approve**	Propose	Consult	-	Consult	-
Implementation approach	Review	**Approve**	Consult	Consult	-	-

Decision Type	Architect	Engineer	Specialist	DevOps	Tester/QA	Instructor
Paradox rule changes	Review	Consult	**Approve**	-	Consult	-
Testing strategy	Consult	Consult	Consult	-	**Approve**	-
Release readiness	Consult	Consult	Consult	Propose	**Approve**	Final
Documentation final	Review	Review	Review	**Approve**	Review	-
Budget (if any)	Propose	Consult	Consult	Consult	-	**Approve**
Timeline adjustments	Consult	Consult	Consult	**Track**	Consult	**Approve**

Communication Boundaries

Internal Communication Matrix

From \ To	Architect	Engineer	Specialist	DevOps	Tester/QA
Architect	-	Design decisions Interface specs	Domain questions Rule validation	Infrastructure needs Deployment requirements	Test requirements Validation criteria
Engineer	Implementation status Technical challenges	-	Paradox implementation Edge cases	Build requests Integration help	Code ready for test Bug fixes
Specialist	Rule impacts Domain updates	Implementation guidance Paradox logic	-	Environment needs Specialist tools	Test scenarios Validation data

From \ To	Architect	Engineer	Specialist	DevOps	Tester/QA
DevOps	Infrastructure status Integration reports	Build status Deployment updates	Environment access Tool support	-	Test environment CI status
Tester/QA	Test results Quality metrics	Bug reports Test coverage	Paradox test results Validation feedback	Test environment needs CI issues	-

Escalation Path

```
None
Issue Discovery
    ↓
Assigned Owner Attempts Resolution (4 hours)
    ↓
Escalate to Team Lead (Architect) (2 hours)
    ↓
Team Discussion/Decision (1 hour)
    ↓
Escalate to Instructor (if unresolved)
```

Boundaries of Responsibility

In-Scope for Each Role

Architect's Boundaries(Vacharapoln)

In Scope:

- System-level design decisions
- All interface definitions
- Technology stack selection
- Architecture documentation
- Design pattern enforcement

Out of Scope:

- Line-by-line code implementation

- Day-to-day coding tasks
- Bug fixing (except critical)
- Test case writing

Engineer's Boundaries

In Scope:

- All core implementation
- Algorithm development
- Performance optimization
- Code documentation
- Technical prototyping

Out of Scope:

- Architecture decisions
- Domain research (beyond implementation)
- Test strategy definition
- CI/CD pipeline maintenance

Specialist's Boundaries

In Scope:

- Paradox rule definitions
- Temporal physics research
- Domain knowledge documentation
- Edge case identification
- Validation scenarios

Out of Scope:

- Core implementation coding
- Infrastructure setup
- Test automation
- Presentation delivery (except domain parts)

DevOps's Boundaries

In Scope:

- Environment setup and maintenance
- CI/CD pipeline
- Version control management
- Build automation
- Integration coordination
- Documentation hosting

Out of Scope:

- Protocol implementation
- Algorithm design
- Test case creation
- Domain research

Tester/QA's Boundaries

In Scope:

- Test strategy and planning
- Test case creation
- Test execution
- Bug tracking and reporting
- Quality metrics
- Release validation

Out of Scope:

- Implementation decisions
- Architecture changes
- Domain rule definition
- Infrastructure setup

Overlap & Handoff Boundaries

Critical Handoff Points

Handoff	From	To	Deliverable	Deadline	Acceptance Criteria
Architecture → Implementation	Architect	Engineer	Interface specs, protocol designs	Week 1 Fri	All interfaces documented, approved
Domain Rules → Implementation	Specialist	Engineer	Paradox rule set, edge cases	Week 2 Mon	5+ rules documented, testable
Code → Testing	Engineer	Tester/QA	Working components, unit tests	Week 2 Fri	Unit tests passing, documented

Handoff	From	To	Deliverable	Deadline	Acceptance Criteria
Components → Integration	Engineer	DevOps	Individual modules	Week 3 Mon	Each runs independently
Integrated System → Validation	DevOps	Tester/QA	Full system	Week 3 Wed	End-to-end flow works
Validated System → Delivery	Tester/QA	All	Release candidate	Week 4 Wed	All tests passing
Final Package → Submission	DevOps	Instructor	All deliverables	Week 4 Fri	Checklist complete

Overlap Zones

Zone	Primary	Supporting	Resolution Mechanism
Protocol Design	Architect	Engineer + Specialist	Design review meeting
Paradox Implementation	Engineer	Specialist	Pair programming
Integration Testing	Tester/QA	DevOps + Engineer	Daily integration standup
Documentation	DevOps	All	Weekly review
Demo Creation	Engineer	All	Demo rehearsal session

RACI Matrix (Responsible, Accountable, Consulted, Informed)

Activity	Architect	Engineer	Specialist	DevOps	Tester/QA
Architecture Definition	**R/A**	C	C	I	I

Activity	Architect	Engineer	Specialist	DevOps	Tester/QA
Interface Design	**R/A**	C	C	-	-
Protocol Specification	**R/A**	C	C	-	-
Implementation	I	**R/A**	C	C	I
Paradox Rules	C	C	**R/A**	-	C
Code Documentation	I	**R/A**	I	C	I
Environment Setup	I	C	-	**R/A**	C
CI/CD Pipeline	-	C	-	**R/A**	C
Unit Testing	I	**R/A**	C	-	C
Integration Testing	C	C	I	C	**R/A**
System Testing	C	I	C	I	**R/A**
Bug Tracking	I	C	I	I	**R/A**
Documentation	C	C	C	**R/A**	C
Release Management	C	C	I	**R/A**	C
Demo Preparation	C	**R/A**	C	C	C
Final Submission	I	I	I	**R/A**	C

Legend:

- **R** = Responsible (Doer)
- **A** = Accountable (The "buck stops here")

- **C** = Consulted (Two-way communication)
- **I** = Informed (One-way communication)

Time Allocation Boundaries

Role	Coding	Design	Testing	Documentation	Meetings	Research
Architect	10%	40%	10%	20%	15%	5%
Engineer	60%	10%	10%	10%	5%	5%
Specialist	20%	20%	20%	15%	10%	15%
DevOps	30%	10%	10%	30%	15%	5%
Tester/QA	10%	20%	40%	15%	10%	5%

Weekly Expected Hours:

- Minimum: 6 hours/week
- Target: 8-10 hours/week
- Maximum: 12 hours/week (sustainable)

Conflict Resolution Boundaries

Conflict Type	Resolution Path	Escalation Time
Technical disagreement	Architect decision	24 hours
Implementation approach	Engineer + Architect consensus	48 hours
Domain rule dispute	Specialist final say	24 hours
Testing priority	Tester/QA prioritization	24 hours
Integration issues	DevOps coordination	12 hours
Schedule conflicts	Team vote → Instructor	48 hours
Scope creep	Architect review → Team vote	24 hours

Sign-off Matrix

Deliverable	Author	Reviewer	Approver
Architecture Specification	Architect	All	Instructor
Implementation Plan	DevOps	All	Architect
Protocol Designs	Architect	Engineer + Specialist	Architect
Source Code	Engineer	Architect + Tester/QA	Engineer
Test Plan	Tester/QA	Engineer	Tester/QA
Test Reports	Tester/QA	All	Tester/QA
User Documentation	DevOps	All	Architect
Demo Video	Engineer	All	DevOps
Final Presentation	All	All	Instructor
Submission Package	DevOps	All	Instructor

Role Boundaries Quick Reference Card

None

```
┌──────────────────────────────────────────────────────────────┐
│                    TEAM BOUNDARIES CHEAT SHEET                 |
├──────────────┬──────────────────────────┬──────────────────────┤
| ROLE         | PRIMARY ZONE             | STAY OUT OF            |
├──────────────┼──────────────────────────┼──────────────────────┤
| ARCHITECT    | Design, Interfaces       | Line-by-line coding    |
├──────────────┼──────────────────────────┼──────────────────────┤
| ENGINEER     | Implementation           | Architecture decisions|
├──────────────┼──────────────────────────┼──────────────────────┤
| SPECIALIST   | Domain rules             | Core coding            |
├──────────────┼──────────────────────────┼──────────────────────┤
| DEVOPS       | Infrastructure           | Algorithm design       |
├──────────────┼──────────────────────────┼──────────────────────┤
```

```
| TESTER/QA    | Testing, Quality        | Implementation         |
└──────────────┴─────────────────────────┴────────────────────────┘
```

This roles, responsibilities, and boundaries table provides clear guidance for all team members, reduces conflict, and ensures accountability throughout the 4-week project. Each team member should review and acknowledge their assigned responsibilities.